CHAPTER 9

BASIC CONCEPTS OF ALGORITHMS

An introduction to some of the basic concepts of computer algorithms for network calculations, particularly data structures for storing networks and methods for estimating the time computations will take

IN THE preceding chapters of this book we have introduced various types of networks encountered in scientific study, methods for collecting data about those networks, and some of the basic theoretical tools used to describe and quantify networks. Then in the last chapter we combined these ideas in an analysis of the structural features of a variety of real-world networks, revealing in the process a number of interesting patterns that will be important to our further studies in the remainder of the book.

Analysis of this kind, and most analysis involved in the contemporary study of networks, is primarily performed using computers. In the early days of network analysis in the first part of the twentieth century, calculations were mostly performed by hand, partly out of necessity, since computers were slow, expensive, and rare, but also because the networks studied were typically quite small, consisting of perhaps just a few dozen vertices or even less. These days we are concerned with networks that have thousands or even millions of vertices. Gathering and analyzing the data for networks like these is only possible because of the advent of fast cheap computing.

Some networks calculations are simple enough that it is obvious how one would get a computer to carry them out, but many are not and performing them efficiently requires careful consideration and thoughtful programming. Even merely storing a network in a computer requires some thought, since there are many methods for doing it and the choice of method can make a substantial difference to the performance of subsequent calculations.

In this chapter and the following two we discuss some of the techniques and algorithms used for network calculations on computers. A good understanding of the material discussed here will form a solid foundation for writing software to perform a wide variety of calculations with network data.

In this chapter we describe some simple but important ideas about the running time of algorithms and data structures for the storage of networks. We will not describe any actual algorithms in this chapter, but the ideas introduced form a foundation for understanding the algorithms that appear in the following chapters.

In Chapter 10 we describe a selection of basic network algorithms, including many of the classics of the field, such as algorithms for calculating centrality indices, finding components, and calculating shortest paths and maximum flows. We continue our study of algorithms in Chapter 11, where we look at matrix-based algorithms and particularly at methods for network "partitioning."

Understanding the content of these chapters does not require that you know how to program a computer. We will not, for instance, discuss particular programming languages. However, some experience with programming will certainly help enormously in understanding the material, and the reader who has none will in any case probably not have very much use for the methods we describe.

Conversely, readers who already have a thorough knowledge of computer algorithms may well find some of the material here too basic for them, particularly the material on run times and data structures in the present chapter. Such readers should feel free to skip material as appropriate and move quickly on to the possibly less familiar subject matter of Chapters 10 and 11. For very advanced readers for whom all the material covered here is already familiar, or readers who simply wish to go into the subject in greater detail, we recommend the books by Cormen *et al.* [81], which is a general computer science text on algorithms, and by Ahuja *et al.* [8], which is specifically on

network algorithms.

Before we leap into the study of algorithms, one further word of advice is worthwhile. Many of the standard algorithms for the study of networks are already available, ready-made, in the form of professional network analysis software packages. Many of these packages are of very high quality, produced by excellent and knowledgeable programmers, and if they are adequate for your needs then there is no reason not to use them. Writing and debugging your own software for the analysis of network data can take hours or days, and there is little reason to expend that time when someone else has already done it for you. Table 9.1 lists some of the most widely used current software packages for the analysis of network data along with a brief description of what they do. The present author, for instance, has made considerable use of Graphviz, Pajek, and yEd, all of which provide useful features that could save you a lot of time in your work. Some other network calculations, especially the matrix-based methods of Chapter 11 and calculations using the models of Chapters 12 to 15, can be performed using standard mathematical software such as Matlab, Mathematica, or Maple, and again there is no reason not to make use of these resources if they are adequate for the particular task before you.

Name	Availability	Platform	Description
Pajek	Free	W	Interactive social network analysis and visualization
Net Workbench	Free	WML	Interactive network analysis and visualization
Netminer	Commercial	W	Interactive social network analysis and visualization
InFlow	Commercial	W	Interactive social network analysis and visualization
UCINET	Commercial	W	Interactive social network analysis
yEd	Free	WML	Interactive visualization
Visone	Free	WL	Interactive visualization
Graphviz	Free	L	Visualization
NetworkX	Free	WML	Interactive network analysis and Python library
JUNG	Free	WML	JAVA library for network analysis and visualization
igraph	Free	WML	C/R/Python libraries for network analysis
GTL	Free	WML	C++ library for network analysis
LEDA/AGD	Commercial	WL	C++ library for network analysis

Table 9.1: A selection of software implementing common network algorithms. Platforms are Microsoft Windows (W), Apple Macintosh (M), and Linux (L). Most Linux programs also run under Unix and Unix-like systems such as BSD, and many Windows programs can run on Macs and Linux systems using emulation software.

That said there are still some excellent reasons for studying network algorithms and computer methods. First of all, even when you are making use of pre-packaged software to do your calculations, it helps greatly if you understand how the algorithms work and what the software is doing. Much time can be wasted when people fail to understand how a program works or misunderstand the kinds of answers the program can give them. Furthermore, if you are going to undertake a substantial amount of work using network data, you will sooner or later find that you need to do something that cannot be done with standard software and you'll have to write some programs of your own.

Second, there is a marked tendency in the current networks literature for some researchers to restrict their calculations to those that can be carried out using the standard software. By relying on pre-packaged programs to do their calculations for them, researchers have become limited in what types of analysis they can perform. In this way, the standard packages have, in effect, shaped the research agenda of the empirical side of the field, which is completely the reverse of what it should be. Good research decides the interesting questions first and then goes in search of answers. Research that restricts itself only to the questions it already knows how to answer will be narrowly

focused indeed. By following the developments in this and the following chapters, and, if you wish, reading further in the algorithms literature, you give yourself the opportunity to pursue whatever network questions are of interest to you, without having to rely on others to produce software to tackle those questions.

9.1 RUNNING TIME AND COMPUTATIONAL COMPLEXITY

Before we can look at exactly how network algorithms work, there is an important issue we need to tackle, that of *computational complexity*. If you have programmed computers before, you may well have had the experience of writing a program to perform a particular calculation and setting it running, only to find that it is still running an hour or even a day later. Performing a quick back-of-the-envelope calculation, you discover to your dismay that the calculation you have started will take a thousand years to finish, and hence that the program you wrote is basically useless.

The concept of computational complexity (or just "complexity" for short) is essentially a more formal version of back-of-the-envelope calculations like this one, and is useful precisely because it helps us to avoid wasting our energies on programs that will not finish running in any reasonable amount time. By considering the complexity of an algorithm before we even start to write a computer program, we can be sure we are writing one that will actually finish.

Computational complexity is a measure of the running time of a computer algorithm. Consider a simple example: how long does it take to find the largest number in a list of n numbers? Assuming the numbers are not given to us in some special order (such as largest first), then there is no quicker way to find the largest than simply to go through the whole list, item by item, keeping a running record of the largest number we have seen, until we get to the end.

This is a very simple example of a computer algorithm. We could use it, for instance, to find the vertex in a network that has the highest degree. The algorithm consists of a number of steps, one for each number in the list. On each step, we examine the next number in the list and ask whether it is larger than the largest we have seen so far. If it is, it becomes the new largest-numberseen-so-far, otherwise nothing happens and we move on to the next step.

Now here is the crucial point: in the *worst possible case* the most work we will have to do for this algorithm is on each step to (1) examine the next number, (2) compare it with our previous record holder, and (3) replace the previous record holder with the new number. That is, the largest amount of work we have to do happens when every number is bigger than all the ones before it.

In this case the amount of work we do is the same on every step and hence the total time taken to complete the algorithm, its running time, is just $n\tau$, where τ is the time taken on each individual step. If we are lucky, the actual time taken may be less than this, but it will never be more. Thus we say that the running time or *time complexity* of this algorithm is order n, or just $O(n)$ for short. Technically the notation $O(n)$ means that the running time varies as a constant times n *or less*, to leading order in n.[127] We say "to leading-order" because it is possible that there may be contributions to the running time that increase with system size more slowly than this leading-order term. For instance, there might be some initial start-up time for the algorithm, such as time taken initializing variables, that is a constant independent of n. We would denote this time as being $O(1)$, i.e., a constant times 1. By convention, however, one drops such sub-leading terms when citing the complexity of an algorithm, because if n is large enough that the running time of the program becomes a serious issue then the sub-leading terms will usually be small enough by comparison with the leading ones that they can be safely neglected.[128] Thus the time complexity of our simple largest-number algorithm is just $O(n)$.

Technically, the computational complexity of an algorithm is an indication of how the algorithm's running time scales *with the size of its input*. In our example, the input to the algorithm is the list of numbers and the size of that input is the length n of the list. If this algorithm were used to find the highest degree node in a network, then the size of the input would be the number of vertices in the network. In many of the network algorithms we will look at this will be the case—the number of vertices n will be the important parameter we consider. In other cases, the important parameter will be the number of edges m in the network, while in others still we will need both m

and n to fully specify the size of the input—there could be different parts to an algorithm, for instance, that operate separately on the vertices and the edges, so that the total running time depends on both. Thus, for example, we will see in Section 10.3 that the algorithm known as "breadth-first search," which is used for finding geodesic paths in networks, has a computational complexity $O(m) + O(n)$ for a network with m edges and n vertices, meaning that it runs in time $am + bn$ where a and b are constants, or quicker. Very often one writes this, in shorthand and slightly sloppy form, as $O(m + n)$. This latter notation is not meant to imply that the constants in front of m and n are the same.

In a lot of networks research we are concerned with sparse graphs (see Section 6.9) and particularly with graphs for which m increases in proportion to n as n becomes large. To put that another way, the mean degree of the network $c = 2m/n$ remains constant (see Eq. (6.23)). In such networks, $O(m + n) \equiv O(n)$ and we can drop the m from our notation.

The importance of the computational complexity lies in its use for estimating the actual running time of real algorithms. If a particular algorithm is going to take a month to solve a problem of interest, or a year or a century, we'd like to know that in advance. We want to estimate how long the calculation is going to take before we start it, so we can make a decision about whether the wait is justified. A knowledge of the computational complexity allows us to do that by measuring run-time on a small problem and then scaling up appropriately to the size of the real problem.

For example, suppose we wish to run the breadth-first search algorithm mentioned above on a network with a million vertices and ten million edges. Knowing that the algorithm has time complexity $O(m + n)$, we could start out with a small test-run of the program on a network with $n = 1000$ vertices, say, and $m = 10\,000$ edges. Often we artificially create small networks just for the purposes of such tests. Perhaps we find that the program finishes in a second on the test network. We then scale up this result knowing that the running time varies as $am + bn$. On the full network with $n = 1\,000\,000$ and $m = 10\,000\,000$ both n and m are a thousand times larger than on the test network, so the program should take about a thousand times longer to finish, i.e., a thousand seconds or about a quarter of an hour. Armed with this information we can safely start our program working on the larger problem and step out for a cup of tea or a phone call while we wait for it finish.

Conversely, suppose we had an algorithm with computational complexity $O(n^4)$. That means that if we increase the number of vertices n in our network by a factor of a thousand the running time will increase by a trillion. In such a case it almost does not matter what the run time of the algorithm is on our small test network; the run time on the full network is going to be prohibitively long. For instance, if the test network takes a second again, then the full network would take a trillion seconds, which is around 30 000 years. In this case, we would certainly abandon the calculation, or at least look for a faster algorithm that can complete it in reasonable time.

Finding the computational complexity of an algorithm, generating test networks, performing small runs, and doing scaling calculations of this type all require some work—additional work on top of the work of developing and programming the computer algorithm in the first place. Nonetheless, this extra work is well worth the effort involved and one should always perform this type of analysis, at least in some rough manner, before embarking on any major numerical calculations. Computational complexity will be one of our major concerns throughout of the discussions of algorithms in this chapter and the following two. An algorithm is next to useless if its running time scales poorly with the size of a network. In practice, any algorithm that scales with system size as $O(n^{129})$ or greater is useless for large networks, although such algorithms still find some use for the smaller cases. In the world of computer science, where many researchers have devoted their entire careers to the invention of new algorithms for solving particular problems, the calculation of the computational complexity of an algorithm is a primary goal—often *the* primary goal—of research. Plenty of papers are published whose sole contribution is to provide a calculation of the complexity of some algorithm.

It is worth mentioning that calculations of the run time of algorithms based on their complexity, as above, do not always give very accurate answers. We have mentioned already that standard measures of time complexity neglect sub-leading contributions to the run time, which may introduce inaccuracies in practical situations. But in addition there are, for technical reasons, many cases where the behavior of the run time is considerably poorer than a simple scaling argument

would suggest. For instance, in calculations on networks it is important that the entire network fit in the main memory (RAM) of our computer if the algorithm is to run quickly. If the network is so large that at least part of it must be stored on a disk or some other slow form of storage, then the performance of the algorithm may be substantially hindered.[129] Even if the entire network fits in the main memory, there may be additional space required for the operation of the algorithm, and that must fit in the memory too. Also, not all kinds of memory are equally fast. Modern computers have a small amount of extra-fast "cache" memory that the computer can use for storing small quantities of frequently used data. If all or most of the data for a calculation fit in the cache, then the program will run far faster than if it does not.

There are also cases in which a program will perform better than the estimate based on its complexity would indicate. In particular, the complexity is usually calculated by considering the behavior of the program in the worst case. But for some programs the worst-case behavior is relatively rare, occurring only for certain special values of the program inputs or particularly unlucky parameter choices, and the typical behavior is significantly better than the worst case. For such programs the complexity can give an unreasonably pessimistic estimate of running time.

For all of these reasons, and some others as well, programs can show unexpected behaviors as the size of their input increases, sometimes slowing down substantially more than we would expect given their theoretical time complexity and sometimes running faster. Nonetheless, computational complexity is still a useful general guide to program performance and an indispensable tool in the computer analysis of large networks.

9.2 STORING NETWORK DATA

The first task of most programs that work with network data is to read the data, usually from a computer file, and store it in some form in the memory of the computer. Network data stored in files can be in any of a large number of different formats, some standard, some not, but typically the file contains an entry containing information for each vertex or for each edge, or sometimes both. The way the data are stored in the computer memory after they are read from the file can, as we will see, make a substantial difference to both the speed of a program and the amount of memory it uses. Here we discuss some of the commonest ways to store network data.

The first step in representing a network in a computer is to label the vertices so that each can be uniquely identified. The most common way of doing this is to give each a numeric label, usually an integer, just as we have been doing in our mathematical treatment of networks in Chapters 6 and 7. In the simplest case, we can number the n vertices of a network by the consecutive integers $i = 1 \ldots n$, although in some cases we might wish to use non-consecutive integers for some reason or to start the numbering from a different point. (For instance, in the C programming language it is conventional for numbering to start at zero and run through $i = 0 \ldots n - 1$.) Most, though not all, file formats for storing networks already specify integer labels for vertices, which may simplify things. For those that don't, one typically just labels vertices consecutively in the order they are read from the file. In what follows, we will assume that vertices are numbered 1 ... n.

Often the vertices in a network have other notations or values attached to them in addition to their integer labels. The vertices in a social network, for instance, might have names; vertices in the World Wide Web might have URLs; vertices on the Internet might have IP addresses or AS numbers. Vertices could also have properties like age, capacity, or weight represented by other numbers, integer or not. All of these other notations and values can be stored straightforwardly in the memory of the computer by defining an array of a suitable type with n elements, one for each vertex, and filling it with the appropriate values in order. For example, we might have an array of n text strings to store the names of the individuals in a social network, and another array of integers to store their ages in years.

Having devised a suitable scheme for storing the properties of vertices, we then need a way to represent the edges in the network. This is where things get more complicated.

9.3 THE ADJACENCY MATRIX

In most of the mathematical developments of previous chapters we have represented networks by their adjacency matrix A_{ij}—see Section 6.2. The adjacency matrix also provides one of the simplest ways to represent a network on a computer. Most computer languages provide two-dimensional arrays that can be used to store an adjacency matrix directly in memory. An array of integers can be used if the adjacency matrix consists only of integers, as it does for unweighted simple graphs or multigraphs. An array of floating-point numbers is needed for an adjacency matrix that may have reals (non-integers) as its elements, as does the adjacency matrix of some weighted networks—see Section 6.3.

Storing a network in the form of an adjacency matrix is convenient in many ways. Most of the formulas and calculations described in this book are written out in terms of adjacency matrices. So if we have that matrix stored in our computer it is usually a trivial matter to turn those formulas into computer code to calculate the corresponding quantities.

The adjacency matrix can be highly advantageous for other reasons too. For instance, if one wishes to add or remove an edge between a given pair of vertices, this can be achieved very quickly with an adjacency matrix. To add an edge between vertices i and j one simply increases the ijth element of the adjacency matrix by one. To remove an edge between the same vertices one decreases the element by one. These operations take a constant amount of time regardless of the size of the network, so their computational complexity is O(1). Similarly if we want to test whether there is an edge between a given pair of vertices i and j we need only inspect the value of the appropriate matrix element, which can also be done in O(1) time.

Undirected networks give a slight twist to the issue since they are represented by symmetric matrices. If we want to add an undirected edge between vertices i and j, then in principle we should increase both the ijth and jith elements of the adjacency matrix by one, but in practice this is a waste of time. A better approach is to update only elements in the upper triangle of the matrix and leave the lower one empty, knowing that its correct value is just the mirror image of the upper triangle.[130] To put this another way, we only update elements (i, j) of the matrix for which $i < j$. (For networks in which self-edges are allowed, we would use the diagonal elements as well, so we would update elements with $i \leq j$—see Section 6.2.) For instance, if we wish to create an edge between vertex 2 and vertex 1, this means in principle that we want to increase both the (2, 1) element and the (1, 2) element of the adjacency matrix by one. But, since we are only updating elements with $i < j$, we would increase only the (1, 2) element and leave the other alone.[131]

Taking this idea one step further, we could not bother to store the lower triangle of the adjacency matrix in memory at all. If we are not going to update it, why waste memory storing it? Unfortunately, dropping the lower triangle of the matrix makes our remaining matrix triangular itself, and most computer languages don't contain arrays designed to hold triangular sets of quantities. One can, by dint of a certain amount of work, arrange to store triangular matrices using, for example, the dynamic memory allocation facilities provided by languages like C and JAVA, but this is only worth the effort if memory space is the limiting factor in performing your calculation.

The adjacency matrix is not always a convenient representation, however. It is cumbersome if, for instance, we want to run quickly through the neighbors of a particular vertex, at least on a sparse graph. The neighbors of vertex i are denoted by non-zero elements in the ith row of the adjacency matrix and to find them all we would have to go through all the elements of the row one by one looking for those that are non-zero. This takes time $O(n)$ (since that is the length of the row), which could be a lot of time in a large network, and yet on a sparse graph most of that time is wasted because each vertex is connected to only a small fraction of the others and most of the

elements in the adjacency matrix are zero. As we will see in this chapter, many network algorithms do indeed require us to find all neighbors of a vertex, often repeatedly, and for such algorithms the adjacency matrix is not an ideal tool.

Operation	Adjacency matrix	Adjacency list	Adjacency tree
Insert	$O(1)$	$O(1)$	$O(\log(m/n))$
Delete	$O(1)$	$O(m/n)$	$O(\log(m/n))$
Find	$O(1)$	$O(m/n)$	$O(\log(m/n))$
Enumerate	$O(n)$	$O(m/n)$	$O(m/n)$

Table 9.2: The leading-order time complexity of four operations for various representations of a network of n vertices and m edges. The operations are adding an edge to the network (insert), removing an edge from the network (delete), testing whether a given pair of vertices are connected by an edge (find), and listing the neighbors of a given vertex (enumerate).

The computational complexity of the network operations discussed here for an adjacency matrix is summarized in Table 9.2.

Another disadvantage of the adjacency matrix representation is that for sparse graphs it makes inefficient use of computer memory. In a network in which most elements of the adjacency matrix are zero, most of the memory occupied by the matrix is used for storing those zeros. As we will see, other representations exist, such as the "adjacency list," that avoid storing the zeros and thereby take up much less space.[132]

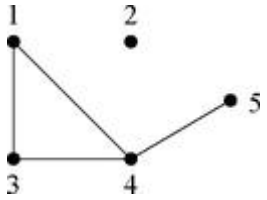
It is a simple matter to work out how much memory is consumed in storing the adjacency matrix of a network. The matrix has n^2 elements. If each of them is an integer (which requires 4 bytes for its storage on most modern computers) then the entire matrix will take $4n^2$ bytes. At the time of writing, a typical computer has about 10^{10} bytes of RAM (10 GB), and hence the largest network that can be stored in adjacency matrix format satisfies $4n^2 = 10^{10}$, or $n = 50\,000$. This is not nearly large enough to store the largest networks of interest today, such as large subsets of the Web graph or large social networks, and is not even big enough for some of the medium-sized ones.

The disadvantages of the adjacency matrix representation described here apply primarily to sparse networks. If one is interested in dense networks—those in which a significant fraction of all possible edges are present—then the adjacency matrix format may be appropriate. It will still use a lot of memory in such cases, but so will any data format, since there is simply a lot of information that needs to be stored, so the advantages of other formats are less significant. The adjacency matrix may also be a good choice if you are only interested in relatively small networks. For instance, the social network analysis package UCINET, which is targeted primarily at sociological practitioners working with smaller networks, uses the adjacency matrix format exclusively. Most current research on networks however is focused on larger data sets, and for these another representation is needed.

9.4 THE ADJACENCY LIST

The simplest alternative to storing the complete adjacency matrix of a network is to use an *adjacency list*. The adjacency list is, in fact, probably the most widely used network representation for computer algorithms.

An adjacency list is actually not just a single list but a set of lists, one for each vertex i. Each list contains the labels of the other vertices to which i is connected by an edge. Thus, for example, this small network:



would be represented by this adjacency list:

Vertex	Neighbors
1	3, 4
2	
3	4, 1
4	5, 1, 3
5	4

An adjacency list can be stored in a series of integer arrays, one for each vertex, or as a two-dimensional array with one row for each vertex.[133] It is common to also store somewhere the degree of each vertex, so that we know how many entries there are in the list of neighbors for each vertex; this can be done using a separate array of n integers. Note also that there is usually no requirement that the neighbors of a vertex in the adjacency list appear in numerical order. Normally they are allowed to appear in any order.

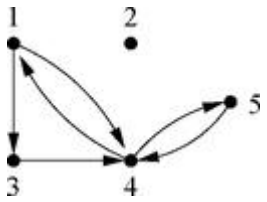
In the example adjacency list above, each edge appears twice. For instance, the existence of an edge between vertices 1 and 3 means that vertex 3 is listed as a neighbor of vertex 1 and vertex 1 is also listed as a neighbor of vertex 3. To represent m edges, therefore, we need to store $2m$ integers. This is much better than the n^2 integers used to store the full adjacency matrix.[134] For instance, on a computer where each integer occupies 4 bytes of memory, a network with $n = 10\,000$ vertices and $m = 100\,000$ edges would occupy 800 kB in adjacency list form, as opposed to 400 MB in matrix format. The double storage of the edges is slightly wasteful—we could save an additional factor of two if we only stored each edge once. However, the double storage turns out to have other advantages, making our algorithms substantially faster and easier to program in many cases, and these benefits are normally worth the extra cost in terms of space. In these days of cheap memory, not many networks are large enough that space to store an adjacency list is a serious consideration.

An adjacency list can store networks with multiedges or self-edges. A multiedge is represented by multiple identical entries in the list of neighbors of a vertex, all pointing to the same adjacent

vertex. A self-edge is represented by an entry identifying a vertex as its own neighbor. In fact, a self-edge is most correctly represented by *two* such entries in the list, so that the total number of entries in the list is still equal to the degree of the vertex. (Recall that a self-edge adds 2 to the degree of the vertex it is connected to.)

The example adjacency list above is for an undirected network, but adjacency lists can be used with directed networks as well. For instance, this network: could be represented by this adjacency list:

Vertex	Outgoing edges
1	3, 4
2	
3	4
4	5, 1
5	4



Here we have listed the outgoing edges for each vertex. Since each edge is outgoing from *some* vertex, this approach is guaranteed to capture every edge in the network, but each edge now appears only once in the adjacency list, not twice as in the undirected case.

Alternatively, we could represent the same network by listing the ingoing edges for each vertex thus:

Vertex	Incoming edges
1	4
2	
3	1
4	3, 1, 5
5	4

In principle these two representations are equivalent. Both include all the edges and either of them can be constructed from a knowledge of the other. When creating computer programs, however, the crucial point is to have the information you need for your calculations easily available, so that the program runs fast. Different calculations require different information and some might need ingoing edges while others need outgoing ones. The choice of which adjacency list to use thus depends on the particular calculations being performed. Some calculations even require both ingoing and outgoing edges, in which case we could create a double adjacency list like this:

Vertex	Incoming edges	Outgoing edges
1	4	3, 4
2		
3	1	4
4	3, 1, 5	5, 1
5	4	4

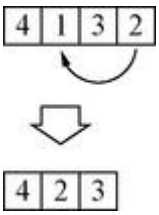
Note that, as in the undirected case considered above, this double adjacency list stores each edge twice, once as an incoming edge and once as an outgoing one, and is thus in some respects wasteful of space, although not to an extent that is normally a problem.

As with the adjacency matrix it is important also to ask how fast our calculations will run if we use an adjacency list. Will they still run at a reasonable pace? If the answer is no, then the adjacency list is not a useful representation, no matter what its other advantages may be.

Consider the undirected case[136] and the four basic network operations that we considered previously for the adjacency matrix, addition and removal of edges, finding whether an edge is present, and enumeration of all edges connected to a vertex—see Table 9.2.

We can add an edge to our network very quickly: to add an edge (i, j) we need only add one new entry each to the ends of the neighbor lists for vertices i and j, which takes time O(1).

Finding or removing an edge is a little harder. To find whether an edge exists between vertices i and j we need to go through the list of neighbors of i to see whether j appears in that list, or vice versa. Since the list of neighbors is in no particular order, there is no quicker way of doing this than simply going through the entire list step by step from the beginning. In the worst case, we will have check all elements to find our edge or to confirm that it does not exist, and on average[137] this will take time of order the mean number c of elements in the list, which is given by the mean degree $c = 2m/n$ (Eq. 6.23). Thus the "find" operation takes time $O(m/n)$ for a network in adjacency list form. This is a bit slower than the same operation using an adjacency matrix, which takes time O(1) (Section 9.3). On a sparse graph with constant mean degree, so that $m \propto n$ (see Sections 6.9 and 9.1), $O(m/n) \equiv O(1)$, so technically the complexity of the adjacency list is as good as that of the adjacency matrix, but in practice the former will be slower than the latter by a constant factor which could become large if the average degree is large.



The element "1" is deleted from a list by moving the last element "2" to overwrite it.

Removing an edge involves first finding it, which takes time $O(m/n)$, and then deleting it. The deletion operation can be achieved in O(1) time by simply moving the last entry in the list of neighbors to overwrite the entry for the deleted edge and decreasing the degree of the vertex by one (see figure). (If there is no last element, then we need do nothing other than decreasing the degree by one.) Thus the leading-order running time for the edge removal operation is $O(m/n)$.

However, the adjacency list really comes into its own when we need to run quickly through the neighbors of a vertex, a common operation in many network calculations, as discussed in Section 9.3. We can do this very easily by simply running through the stored list of neighbors for the

vertex in question, which takes time proportional to the number of neighbors, which on average is $c = 2m/n$. The leading-order time complexity of the operation is thus $O(m/n)$, much better than the $O(n)$ of the adjacency matrix for the same operation.

The computational complexity of operations on the adjacency list is summarized in Table 9.2.

9.5 TREES

The adjacency list is, as we have said, probably the most commonly used format for the storage of networks. Its main disadvantage is the comparatively long time it takes to find or remove edges—$O(m/n)$ time, compared with the $O(1)$ of the adjacency matrix. In many of the most common network algorithms, such as the breadth-first search of Section 10.3 or the augmenting path algorithm of Section 10.5, this is not a problem, since one never needs to perform these operations. Some algorithms, however, such as the algorithm for calculating the clustering coefficient given in Section 10.2, do require these operations and can be slowed by the use of an adjacency list. On a network with mean degree 100, for instance, the edge-finding operations in the clustering coefficient algorithm can be expected to slow the calculation down by about a factor of 100, which could make the difference between a calculation that takes an hour to finish and one that takes a week.

A data structure that has most of the advantages of the adjacency list, while being considerably faster in many situations, is the *adjacency tree*.[138] An adjacency tree is identical to an adjacency list except that each "row"—the set of neighbors of each vertex—is stored in a tree rather than a simple array. If you already know what a tree is then you probably don't need to read the rest of this section—you'll have got the idea already. If you don't know what a tree is, read on.[139]

A tree is a data structure for storing values of some kind, such as integers or real numbers, in a way that allows them to be added, removed, and located quickly. Trees come in a number of types. We consider the case where the values stored are all distinct, which is the relevant case for our purposes, although it is only slightly more complicated to allow for identical values.

Figure 9.1a shows an example of a tree that is being used to store the integers {1, 3, 4, 6, 7, 10, 12, 14}. The basic structure is one of a set of nodes (not to be confused with the nodes of the network that we are storing), which correspond to memory locations in the computer, each containing one of the integers. The tree nodes are arranged in a top-down fashion with a *root node* at the top.[140] Each node can have zero, one, or two *child nodes* that are drawn immediately below it and connected to it by lines to indicate the child relationship. One can create trees with nodes that have more than two children, but for our purposes it works better to limit the number to two. A tree with at most two children per node is called a *binary tree*.

Each node in the tree, except for the root, has exactly one *parent node*. The root has no parent. Nodes with no children are called *leaves*. On a computer a tree is typically implemented with dynamically allocated memory locations to store the numbers, and pointers from one node to another to indicate the child and parent relationships.[141]

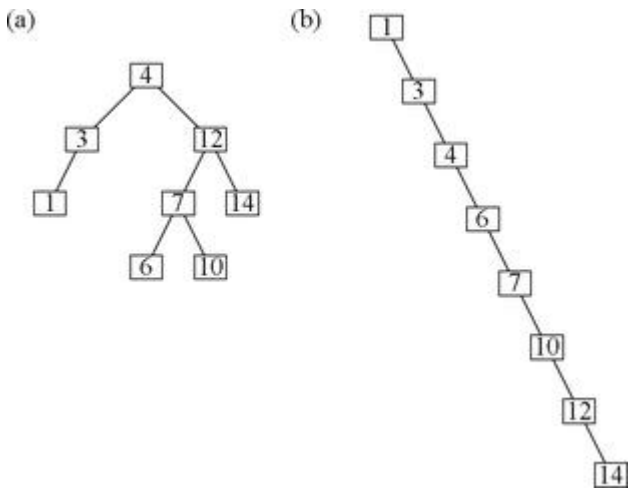


Figure 9.1: Two trees containing the same set of numbers. The two trees depicted here both store the numbers {1, 3, 4, 6, 7, 10, 12, 14}. (a) A balanced tree in which the depths of the subtrees below each node differ by no more than 1. (b) An unbalanced tree.

A defining property of our binary tree will be that the values stored in the left child of node *i* (if there is one) and in all other nodes descended from that child, are *less than* the value stored in *i* itself. Conversely, the values stored in the right child of *i* (if there is one) and all nodes descended from it are *greater than* the value stored in *i*. The reader might like to confirm that the values in Fig. 9.1a satisfy these requirements at every node in the tree.

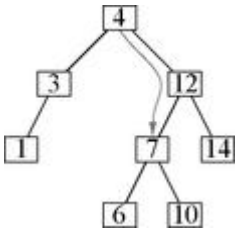
Our goal is to use trees to store the lists of neighbors of each vertex in a network. We will show that if we do so we can perform our four basic network operations—addition, removal, and finding of edges, and enumeration of the complete set of a vertex's neighbors—very quickly on average. Below we first explain in the general language of the binary tree how these operations are achieved. At the end of the section we discuss how they are used in the specific context of the adjacency tree format for networks.

The find operation: The first tree operation we consider is the "find" operation, which is the operation of determining whether a particular value is present in our tree. We accomplish this operation as follows. Starting at the root node:

1. Examine the value *x* in the current node. If it is the value we are looking for, stop—our task is done.
2. If not and the value we are looking for is less than *x*, then by the properties described above, the value we are looking for must be in the left child of the current node or one of its descendants. So we now move to the left child, if there is one, which becomes our new current node. If there is no left child, then the value we are looking for does not exist in the tree and our task is done.
3. Conversely if the value we are looking for is greater than *x*, we move to the right child, if there is one, which becomes our new current node. If there is no right child, the value we are looking for does not exist in the tree and our task is done.
4. Repeat from step 1.

Taking the example of the tree in Fig. 9.1a, suppose that we are trying to determine whether the number 7 appears in the tree. Starting at the root note we find a 4, which is not the number 7 that

we are looking for. Since 7 is greater than 4, we move to the right child of the root and there find a 12. Since 7 is less than 12 we move to the left child and there we find our number 7, and our search is over.



The path taken through our binary tree to locate the number 7.

On the other hand, suppose we wish to determine whether the number 9 appears in the tree. Again we would start at the root and move right, then left, then right as we went down the tree, arriving after three steps at the node containing the number 10. This number is not equal to the number 9 and since 9 is less than 10 we would normally now move to the left child down the tree. In this case, however, there is no left child, and hence we conclude that the number we are looking for does not exist in the tree.

How long does the "find" operation take? That depends on how many steps we have to take through the tree: we have to perform the same operations on each step, so the time taken is simply proportional to the number of steps. The maximum number of steps to reach any node from the root is called the *depth* of the tree. Unfortunately, the depth doesn't have a simple fixed value. There are many possible ways to store the same numbers in a tree while still obeying the conditions on child and parent nodes. For instance, both of the trees in Fig. 9.1 are valid ways to store the same set of numbers. The one in panel (a) has depth 4, while the one in panel (b) has depth 8. In general the maximum possible depth is given by an arrangement like (b) and is equal to the number k of values stored in the tree. For such a tree the find operation would, in the worst case, take O(k) time, which is no better than what we get if we store the values in a simple array—in either case we just end up going through the values one by one until we find the one we want.

On the other hand, if we can make the tree reasonably *balanced*, like the one in Fig. 9.1a, then the depth can be a lot less than k. Let us calculate the minimum depth required to store k values. If we start at the top of the tree and fill in the levels one by one, we can put one value at the root, two values in the second level, four in the third, and so forth, each level holding twice as many values as the previous one. In general, level l can hold 2^{l-1} values and the total number of values in L levels is

$$\sum_{l=1}^{L} 2^{l-1} = 2^L - 1.$$

(9.1)

Setting this equal to k and rearranging, we find that $L = \log_2 (k + 1)$. However, L must be an integer, so, rounding up, we conclude that a minimum of

$$L = \lceil \log_2(k+1) \rceil$$

levels are needed to store k numbers, where $\lceil x \rceil$ denotes the smallest integer not less than x. If we can pack the values into the tree like this, filling each level completely, then our find operation will only take $O(\lceil \log_2(k+1) \rceil)$ to complete. In fact, since $\lceil \log_2(k+1) \rceil \leq \log_2(k+1) + 1$ we could also say that it will take O ($\log_2$ (k + 1)) time, where as usual we have kept only the leading-order term and dropped the sub-leading +1 term. We can also neglect the base of the logarithm, since all logs are proportional to one another, regardless of their base, and we can replace $\log(k + 1)$ by $\log k$ since again we are only interested in the leading-order scaling.

Thus, we conventionally say that the find operation in a balanced tree containing k values can be completed in time $O(\log k)$. This is much better than the $O(k)$ of the simple list or the unbalanced tree. For a tree with $k = 100$ values stored in it we have $\log_2 k \simeq 7$, so the find operation should be about $100/7 \simeq 14$ times faster than for the simple list, and the speed advantage increases further the larger the value of k.

The addition operation: And how do we add a new value to a tree? The crucial point to notice is that in adding a new value we must preserve the relations between parent and child nodes, that lower values are stored in the left child of a node and its descendants and higher ones in the right child and descendants. These relations were crucial to the speedy performance of the find operation above, so we must make sure they are maintained when new items are added to the tree.

This, however, turns out not to be difficult. To add an item to the tree we first perform a "find" operation as above, to check if the value in question already exists in the tree. If the value already exists, then we don't need to add it. (Imagine for example that we are adding an edge to a network. If that edge already exists then we don't need to add it.)

On the other hand, if the value does not exist in the tree then the find operation will work its way down the tree until it gets to the leaf node that *would* have been the parent of our value, but which does not have an appropriate child node. Then we simply add the appropriate child of that leaf node and store our value in it, thereby increasing the number of nodes in the tree by one. Since the find operation takes $O(\log k)$ time and the creation of the new node takes constant time, the leading-order complexity of the addition operation is $O(\log k)$.

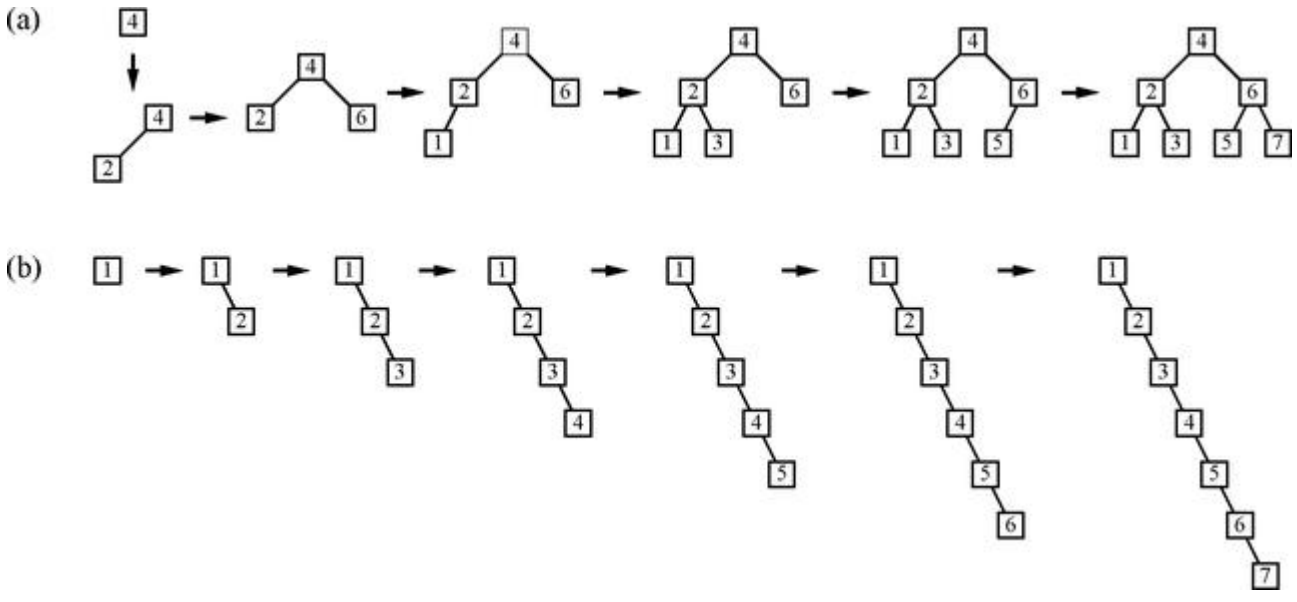
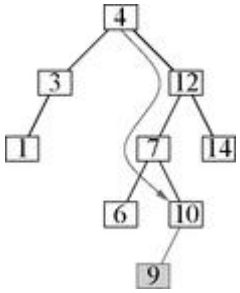


Figure 9.2: The structure of a tree depends on the order in which elements are added to it. Here the values 1 to 7 are added to a tree in two different orders. In (a) they are added in the order 4, 2, 6, 1, 3, 5, 7, resulting in a perfectly balanced tree with the minimum possible depth of three. In (b) they are added in the order 1, 2, 3, 4, 5, 6, 7, resulting in an unbalanced tree with the maximum possible depth of seven.

Balancing the tree: This is satisfying and simple, but it immediately raises another problem. The position a newly added value occupies in the tree depends on the values that are already in the tree, and hence the shape of the tree depends, in general, on the values we add to it and the order in which we add them. Indeed, it turns out a tree can take quite different shapes even for the same values, just as a result of adding those values in different orders. In particular a tree can end up balanced or not as a result of different orders of addition—see Fig. 9.2—and if we are unlucky and get an unbalanced tree then the speed of our tree operations can be severely affected. Obviously we would like to avoid this if possible.



Addition of the number 9 to the tree.

For algorithms in which elements are added to the tree only at the beginning of the program and no elements are added or removed thereafter (the clustering coefficient algorithm of Section 10.2 is an example), a simple solution to the problem is just to randomize the order in which the elements are added. Although the resulting tree is not completely packed full, as in the case considered above, it will still have depth O(log k) on average.[142]

If our algorithm requires us to add or remove values as we go along, then this approach will not work and we must explicitly balance the tree by performing *rebalancing* operations whenever the tree becomes unbalanced. We typically don't attempt to keep the elements packed as tightly in the tree as possible, but we can still achieve O(log k) run times by adopting a looser definition of balance. One practical and simple definition (though not the only one) is that a tree is balanced if the depth of the two subtrees below any node differ by no more than 1. The tree in Fig. 9.1a satisfies this criterion. For instance, the depths of the subtrees of the node containing the value 12 are 2 on the left and 1 on the right.

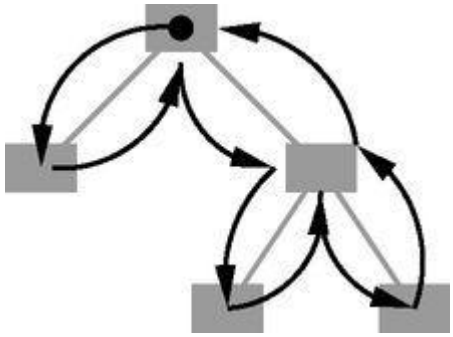
It is straightforward to prove that a tree satisfying this criterion, which is called an *AVL tree*, has depth O(log k). What's more it is possible to maintain this level of balance with fairly simple rebalancing operations called "pivots" that themselves take only O(log k) time to perform. As a result, we can both find elements in and add elements to an AVL tree in time O(log k), even if we have to rebalance the tree after every single addition (although usually we will not have to do this).

Details of the workings of the AVL tree can be found in most books on computer algorithms, such as Cormen *et al.* [81]. However, if you need to use such a tree in your own work it's probably not worth programming it yourself. There are many readily available standard implementations of AVL trees or other balanced trees that will do the job just fine and save you a lot of effort. A

suitable one appears, for instance, in the standard C++ library STL, which is available with every installation of C++.

The deletion operation: The process of deleting a value from a tree is rather involved, but none of the algorithms described in this book require it, so we will not go into it in detail here. The basic idea is that you first find the value you want to delete in the normal way, which takes time O(log k), then delete it and perform a "pivot" operation of the type mentioned above to remove the hole left by the deletion. Each of these operations takes at most time O(log k) and hence the entire deletion can be complete in O(log k) time. The interested reader can find the details in Ref. [81].

Enumerating the items in a tree: We can also quickly run through the items stored in a tree, an operation that takes O(k) time, just as it does for a simple list stored in an array. To do this we use an *Euler tour*, which is a circuit of the tree that visits each node at most twice and each edge exactly twice. An Euler tour starts at the root of the tree and circumnavigates the tree by following its outside edge all the way round (see figure). More precisely, starting from the root we move to the left child of the current node if we haven't already visited it and failing that we move to the right child. If we have already visited both children we move upward to the parent. We repeat these steps until we reach the root again, at which point the tour is complete. Since each edge is traversed twice and there are $k - 1$ edges in a tree with k nodes (see Section 6.7), this immediately tells us we can run through all elements in the tree in O(k) time.



An Euler tour.

Given the above properties of trees, the adjacency tree for a network is now simple to define: we store the list of neighbors of each vertex as values in a binary tree. There is one tree for each vertex, or two for a directed network if we want to store both the incoming edges and the outgoing ones. Edges can be found, added, or deleted from the tree for a given vertex in time O(log k), where k is the degree of the vertex, or O (□log k□) when averaged over all vertices. However, the average of the logs of a set of positive numbers is always less than the log of their average, so a running time of O (□log k□) also implies a running time of O(log□k□) ≡ O(log(m/n)).

The computational complexity of operations on the adjacency tree is summarized in Table 9.2. Comparing our three storage formats for networks, the adjacency matrix, list, and tree, we see that there is no overall winner; the choice of format is going to depend on what we want to do with it. For use in algorithms where we are only going to add and remove edges and check for their existence, the adjacency matrix is the fastest option. On the other hand, in algorithms where we are only adding edges or enumerating the neighbors of vertices, which includes most of the algorithms in this chapter, the adjacency list is the clear winner. The adjacency tree is the fastest format if we need to find or delete edges as well as create them. (Note that log(m/n) is usually much better than

a mere m/n, particularly in the case where the mean degree $c \gg 1$.) In algorithms that only need to enumerate edges and not find, add, or delete them, the adjacency list and adjacency tree are equally good in principle. In practice, however, the additional complexity of the adjacency tree usually makes it somewhat slower and the adjacency list is the format of choice.

It's worth bearing in mind that speed isn't everything, and in particular that the adjacency matrix format uses a prohibitive amount of memory space for networks with large numbers of vertices (see Section 9.3). Overall, as we have said, the adjacency list is the format used most often, but the others certainly have their place and each will be useful to us more than once in the remainder of this book.

9.6 OTHER NETWORK REPRESENTATIONS

We have discussed three ways of representing network data in the memory of a computer. These are probably the most useful simple representations and the ones that you are most likely to need if you write your own programs to analyze networks, but there are a few other representations that it is worth knowing about.

Hybrid matrix/list representations: The representations of Table 9.2 all have their advantages and disadvantages, but none is optimal. In the best of all possible worlds, we would like a data structure that can insert, delete, and find edges in O(1) time and enumerate the O(m/n) neighbors (on average) of a given vertex in O(m/n) time, but none of our representations can do this. It is possible to create a representation that *can* do this, however, if we are willing to sacrifice memory space: we can make a hybrid representation that consists of an adjacency matrix *and* an adjacency list. Non-zero elements in the adjacency matrix, those corresponding to edges, are accompanied by pointers that point to the corresponding elements in the adjacency list. Then we can find whether an edge exists between a specified pair of vertices in O(1) time using the adjacency matrix as usual. And we can enumerate the neighbors of a vertex in O(m/n) time using the adjacency list. We can add an edge in O(1) time since both matrix and list allow this anyway (Table 9.2). And finally, we can delete an edge in O(1) time by first locating it in the adjacency matrix and setting the corresponding element to zero, then following the pointers to the relevant elements of the adjacency list and deleting those too by moving the last element of the list to fill their place.

In terms of time complexity, i.e., scaling of run time with network size, this hybrid data structure is optimal.[143] Its main disadvantage is that it uses even more memory than the ordinary adjacency matrix, and hence is suitable only for relatively small networks, up to a few tens of thousands of vertices on a typical computer at the time of writing. If this is not an issue in your case, however, and speed is, then this hybrid representation may be worth considering.

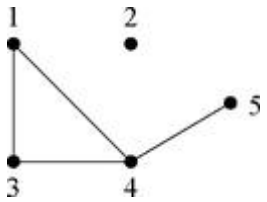
Representations with variables on edges: In some networks the edges have values, weights, or labels on them. One can store additional properties like these using simple variants of the adjacency matrix or adjacency list representations. For instance, if edges come in several types we could define an additional $n \times n$ matrix to go with the adjacency matrix that has elements indicating the type of each extant edge. (For edges that do not exist the elements of such a matrix would have no meaning.) Or one could combine the two matrices into a single one that has a non-zero element for every extant edge whose value indicates the edge type. If there are many different variables associated with each edge, as there are for instance in some social network studies, then one could use many different matrices, one for each variable, or a matrix whose elements are themselves arrays of values or more complicated programming objects like structures. Similarly, with an adjacency list one could replace the elements of the list with arrays or structures that contain all the details of the edges they correspond to.

However, these representations can be wasteful or clumsy. The matrix method can waste huge amounts of memory storing meaningless matrix elements in all the positions corresponding to edges that don't exist. The adjacency list (for an undirected network) contains two entries for each edge, both of which would have to be updated every time we modify the properties of that edge. If each edge has many properties this means a lot of extra work and wasted space.

In some cases, therefore, it is worthwhile to create an additional data structure that stores the properties of the edges separately. For instance, one might use a suitable array of m elements, one

for each edge. This array can be linked to the main representation of the network structure: with an adjacency list we could store a pointer from each entry in the list to the corresponding element in the array of edge data. Then we can immediately find the properties of any edge we encounter in the main adjacency list. Similarly, each entry in the array of edge data could include pointers to the elements in the adjacency list that correspond to the edge in question. This would allow us to go through the array of edge data looking for edges with some particular property and, for example, delete them.

Edge lists: One very simple representation of a network that we have not yet mentioned is the *edge list*. This is simply a list of the labels of pairs of vertices that are connected by edges. Going back to this network, which we saw in Section 9.4:



the edge list representation would be (1, 3), (4, 1), (4, 3), (4, 5). The order of the edges is usually not important in an edge list, nor is the order of the vertices in the vertex pairs.

The edge list is a convenient and space-efficient way to store the structure of a network, and furthermore allows us easily to associate properties with the edges—we can simply store those properties along with the corresponding pairs of labels. It is not such a good representation if we wish to store properties of vertices. Indeed, the representation doesn't explicitly list vertices at all, so there is no way to tell that a vertex even exists if it is not connected to any edges. Vertex 2 in the network above is an example of this problem: it doesn't appear in the edge list because it has no edges. On the other hand, this problem and the problem of storing vertex properties can be remedied easily enough by creating a separate list of vertices and the data associated with them.

However, the edge list is a poor format for storing network data in computer memory for most of the algorithms we will be considering in this book. It does not, for instance, allow us to determine quickly whether a particular edge exists—we would have to go through the entire list to answer that question. And, crucially, it does not allow us easily to enumerate the neighbors of a given vertex, an operation that is central to many algorithms. For these reasons, the edge list is hardly ever used as a format for the representation of a network in memory.

Where it does find use is in file formats for networks. Being a fairly compact representation, edge lists are often used as a way to store network structure in computer files on a disk or other storage medium. When we wish to perform calculations on these networks we must read the file and convert its contents into a more suitable form for calculation, such as an adjacency matrix or adjacency list. This, however, is simple: we create an empty network in the memory of our computer, one with no edges initially, then run through the edges stored in the edge list and add them one by one to the network in the memory. Since the operation of adding an edge can be accomplished quickly in all of the formats we have considered (Table 9.2), this normally does not take a significant amount of time. When it is finished, we have a complete copy of the network in the format of our choice stored in the memory of the computer, and we are ready to continue with our computations.

9.7 HEAPS

The last data structure we describe in this chapter is a specialized structure called a *binary heap*. Unlike the structures introduced in the last few sections, heaps are not normally used for storing networks themselves, but are used for storing values *on* networks, usually values associated with a network's vertices. The definitive property of a heap is that it allows us to quickly find the entry in the heap with the minimum (or maximum) value.

We will make use of the binary heap in Section 10.4 when we study one of the most famous of network algorithms, Dijkstra's algorithm, which is an algorithm for finding shortest paths on weighted networks. This is the main place the heap will come up in this book, so if you are not interested in, or do not need to know, the detailed workings of Dijkstra's algorithm, you can safely skip the remainder of this chapter. Otherwise, read on.

Suppose, then, that we have some numerical value associated with every vertex in a network. That value might be, for instance, the distance from another vertex in the network, or a time until something happens. To give a concrete example, consider a disease spreading across a social network, as discussed in Chapters 1 and 3, and suppose we want to make a computer model of the spread. One simple and efficient way to do this is to associate with each vertex of the network a number representing our current estimate of the time at which that vertex will be infected by the disease (if it ever is). Initially each of these times is set to ∞, except for a single vertex representing the initial carrier of the disease, for which the time is set to zero. Then a simple algorithm for simulating the disease involves at each step finding the next vertex to be infected, i.e., the one with the earliest infection time, infecting it, and then calculating the time until it subsequently infects each of its neighbors. If any of those infection times is earlier than the current recorded time for the same neighbor, the new time supersedes the old one. Then we find the vertex in the network with the next earliest infection time and the process proceeds.

The crucial requirements for this algorithm to run efficiently are that we should be able to quickly find the smallest value of the infection time anywhere on the network and that we should be able to quickly decrease the value at any other given vertex. The binary heap is a data structure that allows us to do these things.

The binary heap is built upon a binary tree structure similar to the trees in Section 9.5, although the tree in a binary heap is arranged and used in a different fashion. Each of the items stored in a heap (items that will represent vertices in the network context) consists of two parts, an integer label that identifies the item and a numerical value. In the disease example above, for instance, the labels are the vertex indices $i = 1 \ldots n$ for vertices not yet infected and the values are the times at which the vertices are infected with the disease.

The items in the heap are stored at the nodes of a binary tree as depicted in Fig. 9.3. There are two important features to notice about this structure. First, the tree is always completely packed. We fill the tree row by row starting with the root node at the top and filling each row from left to right. Thus the tree is denser than the typical binary tree of Section 9.5 and always has a depth that is logarithmic in the number of items in the tree—see Eq. (9.2).

Second, the *values* associated with the items in the heap (the lower number at each node in Fig. 9.3) are *partially ordered*. This means that each value is greater than or equal to the one above it in the tree and less than or equal to both of the two below it. If we follow any branch down the tree—any path from top to bottom—the values grow larger along the branch or stay the same, but never decrease. The values are said to be "partially" ordered because the ordering only applies along branches and not between different branches. That is, there is no special relation between values on different branches of the tree; they may be larger or smaller than one another, whether they are on the same level in the tree or on different levels.

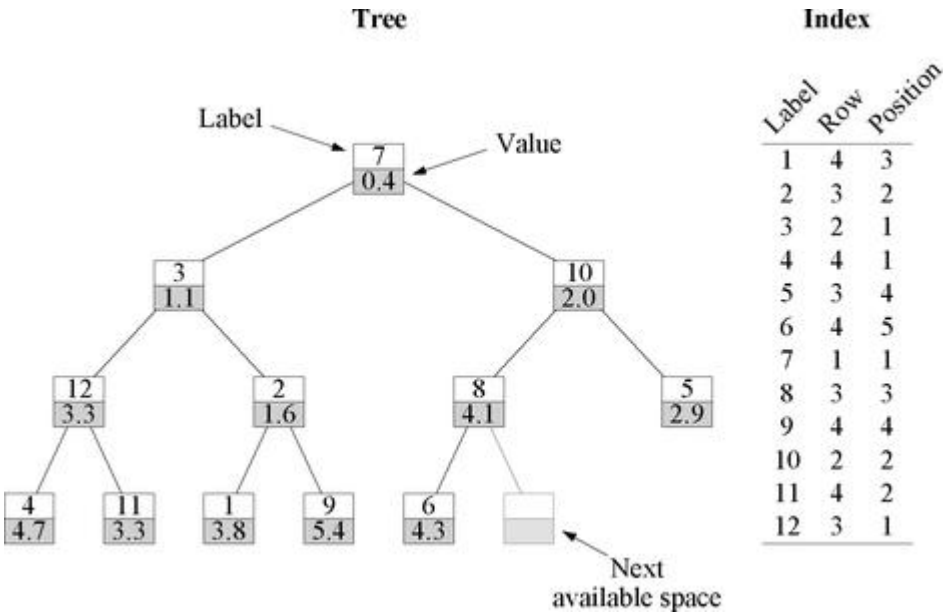


Figure 9.3: The structure of a binary heap. A binary heap consists of two parts, a tree and an index. The nodes of the tree each contain a label and a numerical value, and the tree is partially ordered so that the numerical value at each node is greater than or equal to the value above it in the tree and less than or equal to both of the values below it. The index is a separate array that lists by label the positions of each of the items in the tree, so that we can find a given item quickly. New items are added to the tree in the next available space at the bottom, starting a new row if necessary.

The property of partial ordering has the important result that the value stored at the root node of the tree is always the smallest value anywhere. Since values are non-decreasing along all branches of the tree starting from the root, it follows that no value can be smaller than the root value.

The binary heap also has another component to it, the *index*, which tells us the location of each item in the tree. The index is an array containing the coordinates in the tree of all the items, listed in order of their labels. It might, for instance, contain the row in the tree and position along that row of each item, starting with the item with label 1 and proceeding through each in turn—see Fig. 9.3 again.

A heap allows us to perform three operations on its contents: (1) adding an item to the heap, (2) reducing the numerical value stored in the item with a specified label, and (3) finding and removing the item with the smallest numerical value from the heap. Let us look at how each of these operations is carried out.

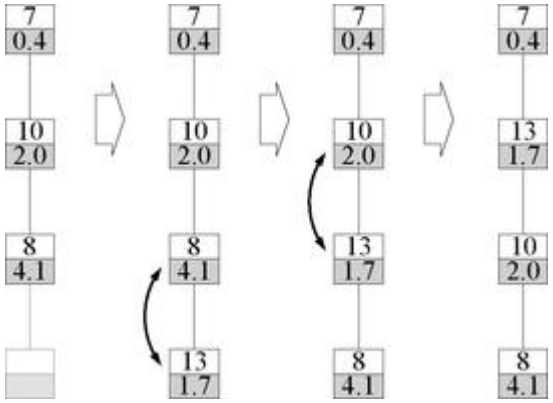


Figure 9.4: Sifting a value up the heap. A branch in the tree initially contains three items as shown. A new item with value 1.7 is added at the bottom. The upward sift repeatedly compares and swaps this value with the value above it until it reaches its correct place in the partial ordering. In this case the added value 1.7 gets swapped twice, ending up (correctly) between the values 0.4 and 2.0.

Adding an item: To add an item to the heap we place it in the first available free space at the bottom of the tree as indicated in Fig. 9.3. If the bottom row of the tree is full, we start a new row. It is important in adding an item to the tree that we preserve the partially ordered structure, but a new item added at the bottom may well violate that structure by having a numerical value smaller than the item above it. We can fix this by *sifting* the tree as illustrated in Fig. 9.4. The newly added item is "sifted up" its branch of the tree by comparing its value with that of the item above it. If its value is smaller, the two are swapped, the new item moving up one row. We repeat this process until the new item either reaches a point where it is not smaller than the item above it, or it has risen to the root of the tree. If it rises to the root of the tree, then by definition it is has the smallest value in the tree because it is smaller than the value for the previous root item.

When we add a new item we also have to update the index of the heap. The coordinates of the new item are recorded at the position in the index corresponding to the item's label and then during the sifting operation, we simply swap the index entries for every two items that are swapped in the tree.

Since the tree is completely packed it has depth given by Eq. (9.2), which is O(log k), where k is the number of items in the tree. Thus the maximum number of swaps we have to do during the sifting process is O(log k) and the total time to add an item to the heap, ignoring sub-leading terms, is also O(log k).

Reducing a value in the heap: Reducing the numerical value associated with a given labeled item is similar to the addition operation. We first use the index to locate the given item in the tree and we reduce its numerical value as desired. Since this may mean that the item is now smaller than one or more of those above it, violating the partial ordering, we sift up as before until the item reaches its correct place in the ordering, updating the index as we go. This operation, like the addition operation, takes time O(log k).

Finding and removing the smallest value: The item in the heap with the smallest numerical value is easily found since, as we have said, it is always located at the root of the tree. In Fig. 9.3, for example, the smallest value is 0.4 for the item with label 7.

We remove the smallest item by first deleting it from the tree and deleting its entry in the index. This leaves a hole in the tree which violates the condition that the tree be completely packed, but we can fix this problem by taking the last item from the bottom row of the tree and moving it up to the root, at the same time updating the relevant entry in the index. This, however, creates its own problem because in moving the item we will likely once again create a violation of the partial ordering of the tree. The item at the root of the tree is supposed to have the smallest numerical value and it's rather unlikely that the item we have moved satisfies this condition. This problem we can fix by "sifting down." Sifting down involves comparing the numerical value stored in the root item with *both* of those below it. If it is larger than either of them, we swap it with the smaller of the two and at the same time swap the corresponding entries in the index. We repeatedly perform such comparisons and swaps, moving our item down the tree until either it reaches a point at which it is smaller than both of the items below it, or it reaches the bottom of the tree.

Again the sifting process, and hence the entire process of removing the root item from the tree, takes time $O(\log k)$.

Thus the binary heap allows us to do all three of our operations—adding an item, reducing a value, or finding and removing the item with the smallest value—in time $O(\log k)$.

PROBLEMS

9.1 What (roughly) is the time complexity of:

a. Vacuuming a carpet if the size of the input to the operation is the number n of square feet of carpet?
b. Finding a word in a (paper) dictionary if the size of the input is the number n of words in the dictionary?

9.2 Suppose you have a sparse network with $m \propto n$. What is the time complexity of:

a. Multiplying an arbitrary n-element vector by the adjacency matrix, if the network is stored in the adjacency matrix format.
b. Performing the same multiplication if the network is in adjacency list format.
c. The "modularity matrix" **B** of a network is the matrix with elements

$$B_{ij} = A_{ij} - \frac{k_i k_j}{2m}.$$

(See Eq. (7.70) on page 224.) What is the time complexity of multiplying an arbitrary vector by the modularity matrix of our sparse network if the network is in adjacency list format?
d. In fact, if we are clever about it, this last operation can be performed in time $O(n)$ for the sparse network with $m \propto n$. Describe an algorithm that achieves this.

9.3 An interesting question, which is discussed in some detail in Chapter 16, concerns what happens to a network if you disable or remove its vertices one by one. The question is of relevance, for instance, to the vaccination of populations against the spread of disease. One typical approach is to remove vertices in order of their degrees, starting with the highest degrees first. Note that once you remove one vertex (along with its associated edges) the degrees of some of the other vertices may change.

In most cases it is not possible to do the experiment of removing vertices from a real network to see what effect it has, but we can simulate the process on a computer by taking a network stored in computer memory, removing its vertices, and then measuring various properties of the remaining network.

a. What is the time complexity of finding the highest-degree vertex in a network, assuming the vertices are given to you in no particular order?
b. If we perform the repeated vertex removal in a dumb way, searching exhaustively for the highest-degree vertex, removing it, then searching for the next highest, and so forth, what is the time complexity of the entire operation?
c. Describe how the same operation could be performed with the degrees of the vertices stored instead in a heap. You will need to modify the heap structure of Section 9.7 in a couple of ways to make the algorithm work. One modification is trivial: the heap needs to

be sorted in the opposite order so that the largest element is at the root. What other modification is needed, and how would you do it? What now is the time complexity of the entire calculation?

d. Taking the same approach, describe in a sentence or two a method for taking n numbers in random order and sorting them into decreasing order using a heap. Show that the time complexity of this sorting algorithm is $O(n \log n)$.

e. The degrees of the vertices in a simple graph are integers between zero and n. It is possible to sort such a set of integers into numerical order, either increasing or decreasing, in time $O(n)$. Describe briefly an algorithm that achieves this feat.